Pattern-Based Development of User-Friendly Web Applications

Ina Wentzlaff
University Duisburg-Essen
Faculty of Engineering
Department of Computer Science
Software Engineering
Oststraße 99
47057 Duisburg, Germany
ina.wentzlaff@uni-due.de

Markus Specker
University Duisburg-Essen
Faculty of Engineering
Department of Computer Science
Interactive Systems and Interaction Design
Lotharstraße 65
47057 Duisburg, Germany
specker@interactivesystems.info

ABSTRACT
Using patterns to preserve common problemsolving knowledge is a very popular approach. Each computer science community applies its own techniques to document and reuse engineering principles about how to handle recurring software development questions with the help of patterns. Thus various pattern collections have evolved to deal with specific problems of the corresponding community. As a result there are comparable patterns or redundant pattern descriptions in different collections leading unintentionally to a "reinvention of the wheel" time and again. Taking the development of a chat application as an example we present an approach for integrating patterns from different software engineering disciplines. We transform problem patterns (*Problem Frames*) into solution patterns (*Design Patterns*) to achieve a pattern-based software development process which systematically leads from natural-language requirements to semi-formal near code level descriptions. Especially, we consider non-functional software properties by combining design patterns of human computer interaction (HCI) with software engineering (SE) patterns to support the methodological development of user-friendly software applications.

Keywords
Software Engineering, Human Computer Interaction, Problem Frames, Design Patterns, Usability.

1. INTRODUCTION
Patterns are abstract descriptions of recurring principles reflecting factual and experience knowledge. A pattern can be applied in different situations to solve concrete engineering problems. Hence pattern-based development techniques have an important contribution to the reuse of software engineering knowledge.

Applying patterns to software design was leveraged by Gamma et al. [10] taking the idea of Alexander et al. [2] who developed design patterns for building architectures and town planning. Meanwhile pattern-based approaches are established in many computer science domains. For instance in software engineering patterns are used to describe software development problems (*Problem Frames* [11]), and analysis tasks (*Analysis Patterns* [9]). Architectural styles and further patterns for software design (*Design Patterns* [5]), human computer interaction problems (*HCI Design Patterns* [17], [4]), and patterns to solve game design questions (*Game Design Patterns* [3]), etc. already exist.

Often the same pattern can be found in several collections with variable names, representations, or classifications [16]. For instance comparing the HCI design pattern "*(Multi Level) Undo*" [17], [18], to software engineering design patterns an adequate SE pattern can be composed out of "*Memento*" and "*Command*" [10]. These widespread redundand or comparable pattern descriptions indicate the possible integration of the different pattern collections. We will make use out of this finding to support a software development process based on systematic pattern transformations, exclusively.

A further important keynote is the way of how to describe patterns. Both laymen and experts ought to be in the position to understand and successfully apply patterns to build usable artifacts with them. Therefore we share the concept of Tidwell [17] who strives to describe HCI design patterns in a layman-understandable way. But like Folmer et al. [8] and Sinnig et al. [15] we see the necessity to extend informal pattern descriptions by technical details. Thus we will finally come to a concrete, near code description that implements a solution of the problem to which the general, informal pattern was applied to.

2. SOFTWARE AND USABILITY ENGINEERING
Various development techniques and methods for constructing software are known from SE which mainly bases on referring functional aspects. Many of these techniques are also used in Usability Engineering (UE). However, especially in

UE there is an urgent need to consider non-functional software properties like *usability* systematically in the software development process (besides style guides and standards for software ergonomics). Actually, only few approaches [7] exist which explicitly take non-functional properties (*soft goals*) into consideration. Nevertheless it is commonly accepted that software quality is mainly reflected by non-functional properties [13].

In this regard patterns in general represent no exception. They usually describe recurring software development principles in a mere functional way. But they can also be assigned with non-functional characteristics: for instance architectural styles can be classified according to software qualities like adaptability, performance, etc. Cf. [14]. HCI design patterns imply non-functional software properties like usability. In the following we combine HCI design patterns with Problem Frames to integrate usability aspects into the software development process.

Problem Frames (PF) are patterns developed by Jackson [11] which can be used for describing problems in the analysis phase of software engineering (*Requirements Engineering*). They represent functional software requirements by documenting necessary interactions between the *"machine"* (the software which is to be build) and its environment in a frame diagram. On the one hand software development problems modeled with Problem Frames can easily be specified and realized with more common software engineering techniques (e.g. UML[1] notations), but on the other hand they support a more explicit and natural representation of requirements which is also supported by our pattern transformation approach.

We extend some Problem Frames by problem descriptions of HCI design patterns and thus obtain HCI-oriented Problem Frames. In the course of the software engineering process we can integrate HCI design patterns systematically and explicitly. Finally, we merge these HCI-oriented problem patterns with patterns describing a solution (like software architectural styles and design patterns). By systematically transforming one (problem) pattern into another (solution) pattern we achieve a pattern-based software development process.

For instance we demonstrate the effectiveness of this approach to systematically construct pattern-based, user-friendly software applications for the development of a chat application.

2.1 Cased-based reasoning to systematically transfrom problem into solution patterns

Cased-based reasoning (CBR) is a problem solving paradigm originated from knowledge engineering [1]. In CBR a *case* consists of a known problem and a corresponding solution.

A *case base* is used to collect these cases. The process of CBR often refered as the R^4-cycle consists of the following steps.

[1]Unified Modeling Language

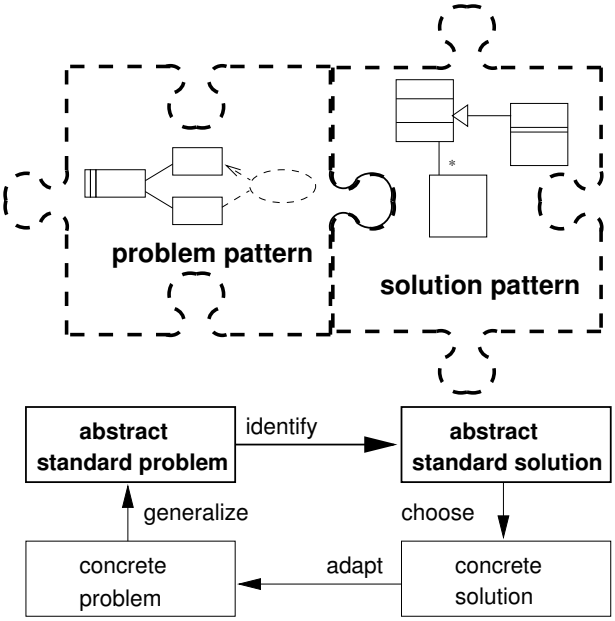


Figure 1: Pattern integration with CBR

Retrieve: A new, concrete problem can be solved with the help of CBR by tracking it back to a problem description which is part of a case in the case base.

Reuse: The solutions described in the identified cases are considered to solve the problem.

Revise: The most appropriate solution will be chosen and adapted to a concrete solution of the given problem.

Retain: The new, adapted solution together with its problem will become a new case which is stored into the case base.

We use CBR to implement a pattern-based software engineering process. A case is a composition of a Problem Frame (as a standard problem) and an architectural style or design pattern (as a standard solution), Cf. Fig. 1. A concrete software development problem is represented by instantiating a Problem Frame. By this a given problem is generalized via a problem pattern into a standard problem. Corresponding solution patterns (architectural styles or design patterns) will be identified and instantiated with the content of the Problem Frame. Thus the instances of the solution patterns represent a concrete solution of a given problem where the instance which fits the given problem in the best manner is chosen and adapted if necessary.

Thereby we shift the pattern-based software development process to the level of standard problems and related standard solutions.

3. DEVELOPING A CHAT APPLICATION

In order to show how user-friendly software can be developed systematically with our suggested pattern-based methodology we start with the definition of informal, natural-language requirements for the chat application which is to be build.

We detail the usage of Problem Frames and how they are related with non-functional software properties in the context of human computer interaction design, before we introduce the transformation of problem into solution patterns.

3.1 Understanding the problem situation

The *system mission* of the chat application can be outlined as follows:

> *"A text-message-based communication platform shall be developed which allows multi-user communication via private I/O-devices."*

To describe this software development goal more precise we derive further requirements `R1`-`R6` as optative statements in Tab. 1. The *requirements* describe wished properties of the environment, after the machine is integrated [11]. Moreover *domain knowledge* which is subdivided into *facts* `F1` and *assumptions* `A1` is recorded. Facts represent fixed characteristics of the application domain, whereas assumptions describe constraints on the application environment. Both, facts and assumptions are indicative statements which are needed to realize the requirements during the implementation process.

`R1`	All users can phrase text-messages.
`R2`	The phrased text-messages are presented on a graphical display.
`R3`	Users send their phrased text-messages to the chat in order to take part in the chat.
`R4`	Text-messages taking part in the chat are registered to the course of chat in their correct temporal order.
`R5`	The course of chat is presented on the graphical display.
`R6`	Sending text-messages changes the graphical display presentation of the course of chat.
`F1`	Users can only understand the course of chat, if the text-messages are presented in their correct temporal order (First In - First Out (*FIFO*)).
`A1`	Users will follow the course of chat on the display.

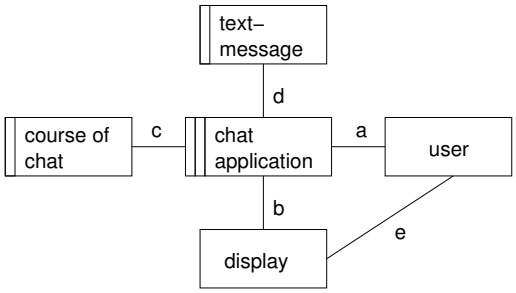
Table 1: Requirements and domain knowledge for the chat application

In our application example we assume that the requirements, and the domain knowledge are completely recorded.

3.1.1 A context diagram for the problem situation

Fig. 2 shows a *context diagram* [11] which represents the overall problem situation by interactions between the machine (the software which is to be developed) and its (application) environment. Knowledge extracted from the requirements, and domain knowledge (Cf. Tab. 1) is represented within the context diagram with *domains* (rectangles) and *shared phenomena* (labeled links between the domains).

The domains correspond to entities of the real world and can be divided into categories as follows: the *machine domain* (rectangle with two vertical lines) represents the software product which ought to be developed, in our case: the **chat application** itself. Data structures, database shema,



a: {phraseTextmessage,sendTextmessage}
b: {showTextmessage,showCourseOfChat}
c: {registerTextmessage,CourseOfChat}
d: {editTextmessage,Textmessage}
e: {followCourseOfChat}

Figure 2: Context diagram for the chat application

or other representations of information relevant to the problem to be solved and needed to be represented by the machine are build by the software developer and centralised in *designed domains* (rectangle with only one vertical line), e.g. here the **text-message**. *Given domains* (simple rectangular) are concepts of the real world which already exist and do not need to be constructed artificially. They are relevant for the problem description and its solution and need to be considered in the context diagram, too, in this case the **display**.

Shared phenomena are operations, actions, events, or states which combine two domains with each other. For instance the machine domain **chat application** shares the phenomena **c**: {"register text message" (`R4`) and "content of course of chat" (`R3`-`R6`)} with the designed domain **course of chat**.

3.2 Decomposing the problem situtation with Problem Frames

Now we decompose the context diagram by mean of knowledge-based projection into smaller, parallel subproblems. Those simple and independent subproblems are represented by instantiated Problem Frames. Whereas for each subproblem all other subproblems will be assumed as already solved (*separation of concerns*). The subproblems are derived from the context diagram using operators for decomposing problems (e.g. by combining domains or omitting shared phenomena).

Fig. 3 shows the instance of the Problem Frame *"commanded workpiece display"*. This frame diagram is a variant of the "*commanded display*" frame developed by Jackson [11] and an enhancement of the "*update*" frame developed by Choppy and Heisel [6]. Compared with the context diagram in Fig. 2 this frame instance contains only domains and shared phenomena which are relevant to fulfill the requirements `R1` and `R2` (Cf. Tab. 1).

The ellipsoid in Fig. 3 maps the requirements (`R1`-`R2`) to the problem context (dashed lines to the domains). The left part of the frame diagram relates the requirements via domains to the machine (the software which has to be developed). Only shared phenomena which are directly related with the

machine can be implemented to realize the requirements.

Arrowheads at dashed lines put constraints on the corresponding domain (concerning behavior or characteristics of this domain stated in the requirements). For instance the requirement `R1` *"users phrase text-message"* describes a restriction on the domain **text-message**, because only the user decides on its content.

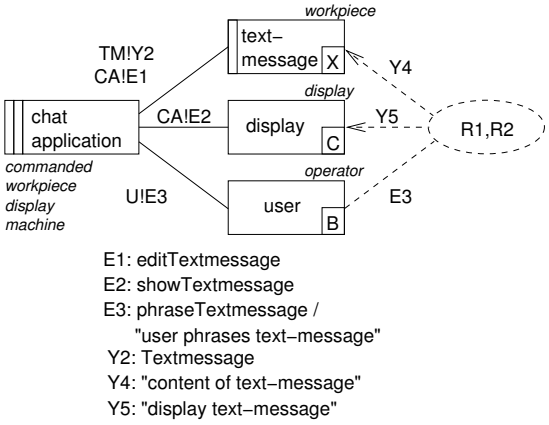


Figure 3: PF "commanded workpiece display"

The characters `Y` and `E` characterize shared phenomena. Symbolic values are indicated by an `Y`, events are annotated with an `E`. The characters are numbered for indexing the shared phenomena. The letters `X`, `C`, and `B` at the domains characterize the domain type. Thus the designed domain **text-message** is a *lexical domain* `X` which works up the symbolic phenomena. The **display** is a given *causal domain* `C` which do not need to build up new but can be controlled according to our wishes. The **user** is represented by a *biddable domain* symbolized by the charcater `B`. His behavior can not be predicted or controlled by the machine. Indeed the user can make inputs to the software, but cannot be forced with requirements to act in a predetermined way. However assumptions as part of the domain knowledge are used to record the assumed user behavior.

The exclamation mark (`!`) at the shared phenomena denote which domain controls a shared phenomenon. However this does not imply control flow. The notation `TM!Y2` in Fig. 3 in fact expresses that the domain **text-message** `TM` is responsible for administrating the symbolic phenomenon `Y2` *"content of text-message"*. Requesting the content of the text-message is controlled by the machine domain.

The Problem Frame *"commanded workpiece display"* in Fig. 3 describes the following problem situation: If the *operator* (**user**) sends a command to the machine (`E3`: "phrase text-message") he disposes the machine to change the appropriate *workpiece* (**text message**) according to the command (`E3` implies `E1`). The machine can follow the state changes of the workpiece (text message) (`Y2`: "content of text-message") and represent them on a *display* (**display**) (through `E2`: "showTextmessage").

Besides this first subproblem represented by the Problem Frame *"commanded workpiece display"* in Fig. 3 there are further instantiated Problem Frames covering the remaining requirements. For the sake of completeness we only draft them in Fig. 4 and Fig. 5.

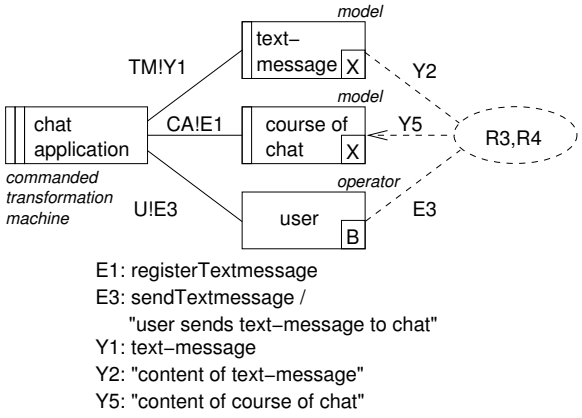


Figure 4: PF "commanded transformation"

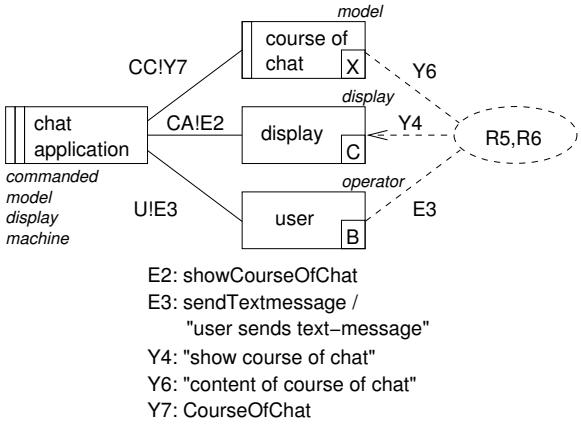


Figure 5: PF "commanded model display"

3.2.1 Considering usability aspects with HCI-oriented problem descriptions

Considering HCI design patterns in our pattern-based software development approach we now have to detail the subproblems which we have already found to reflect usability aspects. For this we adopt problem descriptions of HCI design patterns from Tidwell [17] and van Welie [18] into the Problem Frames approach of Jackson [11].

Fig. 6 extends the standard problem "*commanded workpiece display*" (Fig. 3) by the HCI design patterns **input hints**: *"place a sentence to explain what is required"* (workpiece meta information) and **input prompt**: *"prefills telling the user what to do"* from Tidwell [17]. The HCI-oriented Problem Frame in Fig. 6 describes the following situation: if the *operator* (**user**) has not commanded the *machine* (**chat application**) so far the *workpiece default* (**standard message**, e.g. `<input your text message here.>`) is displayed on the **display** instead of the *workpiece* **text-message** which still is to phrase by the user. In addition to the workpiece (text-message) a *workpiece meta information* (description of the kind of expected inputs, e.g. the caption "`Your text-message:`") is displayed. Besides the two new domains

workpiece default and *workpiece meta information* the behavior of this HCI-oriented Problem Frame does not differ from the problem description of the Problem Frame *"commanded workpiece display"* in Fig. 3.

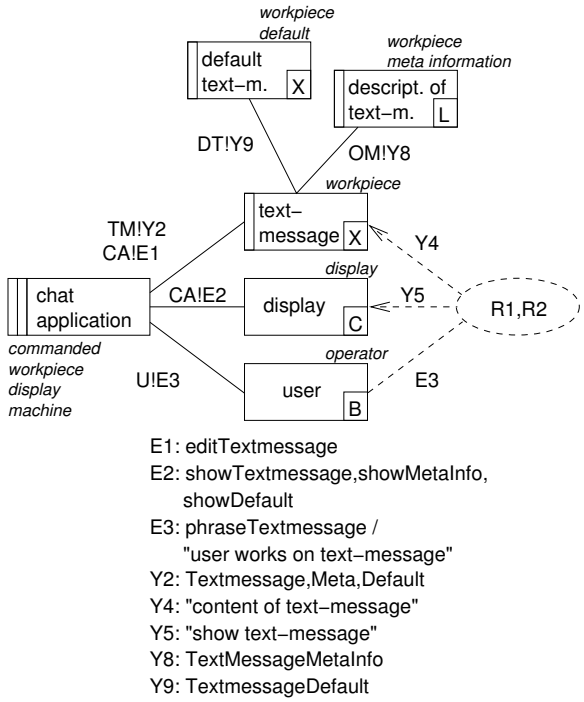


Figure 6: HCI-oriented variant of the PF "commanded workpiece display", Cf. Fig. 3

Fig. 7 shows the HCI-oriented extension of the standard problem *"commanded transformation"* of Fig. 4 by the HCI design pattern **progress**:*"tell user whether or not an operation is still performed and how long it will take"* from van Welie [18].

The HCI-oriented Problem Frame *"commanded transformation"* in Fig. 7 extends the description of the standard problem situation (Cf. Fig. 4) as follows: initiated by the `E3`-command of an *operator* (**user**) an internal machine working process (`Y1` to `E1`) is initialised. On a *transformation display* (**display**) the operator can follow that the *machine* which is still processing a working process (`E2`). The operator need not to know how the working processes are implemented, but he can initialize and observe them.

By integrating problem descriptions of HCI design patterns within Problem Frames it is possible that software quality attributes like usability already can be considered in the early stages of the software development process. The basic functionality of the chat application covered in the Problem Frames of Fig. 3 up to Fig. 5 is improved by usability aspects represented in the HCI-oriented Problem Frames in Fig. 6 and Fig. 7 (by offering more feedback through input hints, input prompts and indication of internal machine working processes to the user). Identifying non-functional software attributes like usability and its corresponding functions already in an early software development phase provides a valuable base for developing user-friendly quality software systematically.

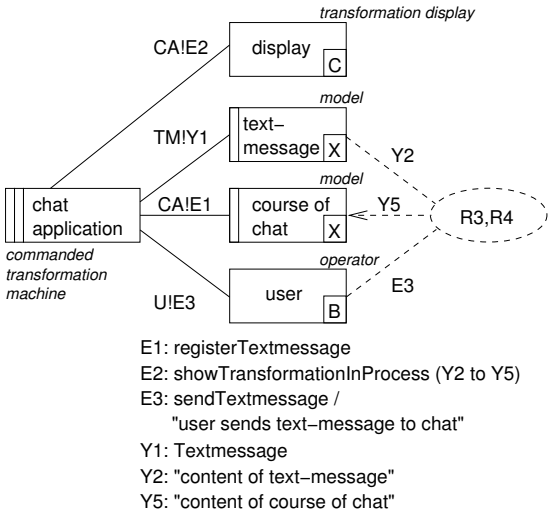


Figure 7: HCI-oriented variant of the PF "commanded transformation", Cf. Fig. 5

The analysis of the problem situation is now completed. In the next step we transform the problem we found into solution patterns which we want to support by CBR in the future. However the descriptions of the new HCI-oriented problem patterns are kept informal anymore that they are understandable for laymen as well. Software requirements described in a natural-language style can be simply divided into different domains and shared phenomena and also graphically represented as Problem Frames to emphasize important entities and relations which have to be considered for describing a problem situation.

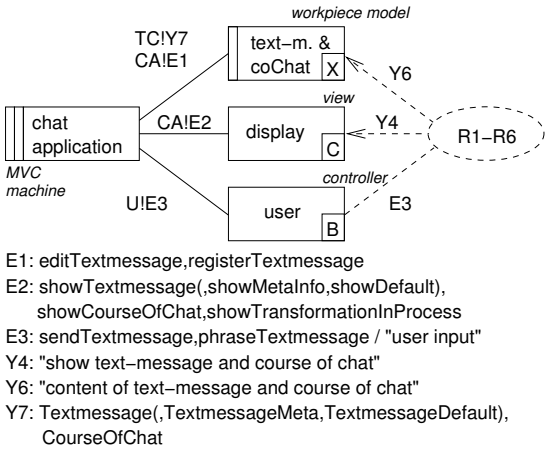


Figure 8: Problem Frame for the architectural style *"model-view-controller"*

4. DESIGN OF THE PROBLEM SOLUTION

Transforming problem patterns like Problem Frames into solution patterns like architectural styles or design patterns facilitates a systematical derivation of near code descriptions only based on natural-language software requirements. The content of an instantiated Problem Frame is used to assign concrete values to the corresponding solution pattern which results in a more detailed specification of an appropriate solution of the given problem.

Similar to Rapanotti et al. [12] we introduce a new Problem Frame in Fig. 8 which reflects the architectural style of Buschmann et al. [5] named *"model-view-controller"*. Taking the domains and shared phenomena of the Problem Frames in Fig. 5, Fig. 6 and Fig. 7 we instantiate the Problem Frame in Fig. 8 which sums their property and behavior descriptions. The *"model-view-controller"* Problem Frame in Fig. 8 can be mapped to the UML class diagram of the corresponding architectural style without any changes on the original patterns, as shown in Fig. 9.

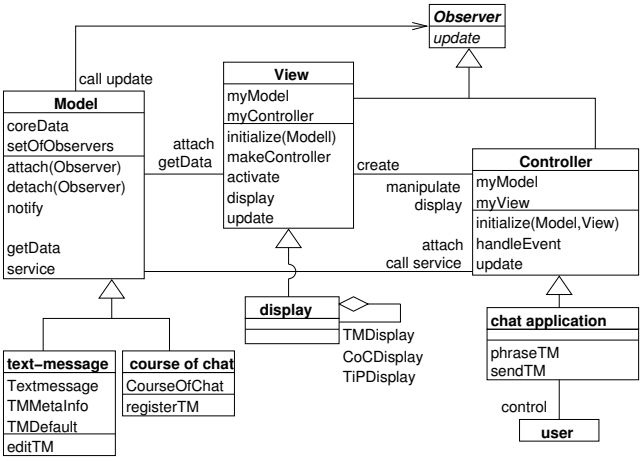


Figure 9: Architectural style *"model-view-controller"* assigned with content of the Problem Frame in Fig. 8

4.1 Transformation of Problem Frames into UML class diagrams

All requirements `R1-R6` of Tab. 1 are covered by the UML class diagram in Fig. 9. This class diagram is an instantiated solution pattern which contributes to solve the given software development problem

It is possible that the domains of the Problem Frame in Fig. 8 are related to the class diagram by an inheritance relation which is based on their role (if they were instances of model, view, or controller) in the Problem Frame. For example the domains **text-message** and **course of chat** are a kind of **Model** in the UML class diagram (as well as in the Problem Frame) providing different services and work on different sets of core data (class attributes).

Shared phenomena controlled by the machine become operations of the corresponding domains. For instance the shared phenomenon `E1`: *editTextmessage* in the subproblems described in Fig. 3 and Fig. 6 becomes the operation *service()* of the class **Model** in Fig. 9 which is implemented by the operation *editTM()* of the inheriting class **text-message**. The shared phenomena `E2`: *showTextmessage*, *showCourseOfChat*, and *showTransformationInProcess* in Fig. 8 are not explicitly stated as operations of the class **display**, because they are only different realizations of the *display()* operation of the class **View**. Our assumption is that the overall display which is readable by the user consists of several subdisplays. These might handle different aspects which should be displayed like showing the text-message, showing the actual course of chat, etc. This assumption does not contradict our systematic pattern transformation approach.

Biddable domains cannot be controlled by the machine, but they can request or initialize services. Thus shared phenomena controlled by a biddable domain (here: the **user**) are assigned to services of the machine, here: the **chat application** itself. The user controls phrasing or sending text-messages of the chat application which treats these requests as a kind of **Controller**.

Shared phenomena that are controlled by lexical or causal domains are simple attributes of the corresponding classes. For instance the shared phenomenon `Y7`: *CourseOfChat* in Fig. 8 which is controlled by the class **course of chat** is an attribute of the class **course of chat** which stores the content of the actual course of chat.

4.2 Informal HCI design patterns as part of the semi-formal design solution

Fig. 9 reflects a coarse-grained design solution for the software development problems specified in Fig. 5 to Fig. 8. By combining the informal HCI design pattern problem descriptions of Tidwell [17] and van Welie [18] with Problem Frames which are used in the requirements engineering phase of the software development process we now can systematically state where and how to apply design patterns and realize software quality aspects like usability.

In contrast to Sinnig et al. [15] who transform HCI design pattern into UML class diagrams, we do not need explicit UML representations of single HCI design patterns. Our approach has the benefits as follows. First we can reuse the natural-language descriptions of the original problem descriptions of HCI design patterns in the analysis phase of the software development process and thus support both laymen and experts working with patterns. Second we are nevertheless in the position to say systematically where and how HCI design patterns should be applied to realize the software product. HCI-oriented Problem Frame can show explicitly which part of the software is effected by usability aspects. By transforming HCI-oriented problem patterns into solution patterns like architectural styles or design patterns, the designer is guided on which place design patterns can be applied instead of introducing them artificially based on personal experience. Thus our approach supports identifying design solutions by giving hints how and where to apply design patterns.

In Fig. 9 the HCI design patterns *"input prompt"*, *"input hint"*, and *"progress"* are part of the designed solution. The class **text-message** has two new attributes `TMMetaInfo` and `TMDefault` which are derived from the corresponding HCI-

oriented Problem Frame in Fig. 6. If the operation *display()* of the class **View** is called, the **TMDisplay** shows the content of the `TextMessage` of the class **text-message** as well as the content of the attributes `TMMetaInfo` (possibly as a label of the text-message) and `TMDefault` (for example as an initial value of the `TextMessage` content). To form an idea of how `TMDisplay` could look like in real, the reader can imagine a labeled text field widget which allows the manipulation of a text-message.

4.3 Applying CBR to identify appropriate problem solutions

One objective of applying case-based reasoning to our pattern-based software development process is to identify different possible solution patterns which fit the given problem situation and use the most appropriate one(s) to realize the problem solution. Rating of different design alternatives with the help of metrics and CBR can be one approach. We use several solution patterns and combine them into an overall design. Until now in the application example only human-computer-interaction problems are considered. They belong to a problem class which often matches with a *"model-view-controller"* structure. Further investigations of the problem situation are necessary to understand the communication problem in more detail that need be solved by our chat application (Cf. system mission in Sec. 3.1).

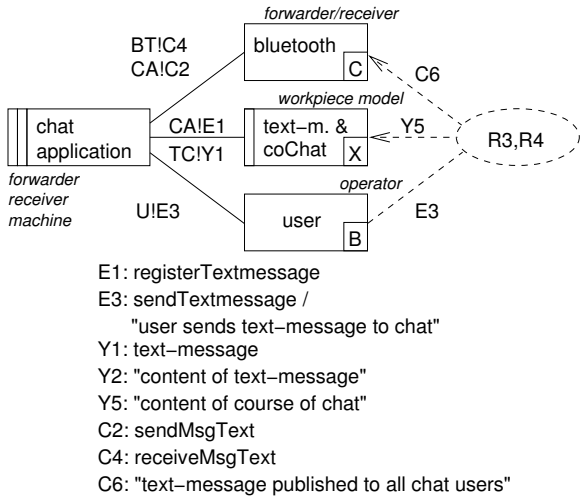


Figure 10: Design Pattern *"forwarder-receiver"* assigned with content of the PF in Fig. 4

Fig. 10 shows the instantiation of the Problem Frame *"forwarder-receiver"* with the Problem Frame *"commanded transformation"* in Fig. 4. The shared phenomena `sendTextmessage` and its related phenomena `registerTextmessage` are responsible to publish the private phrased text-messages to all users of the chat. The *"forwarder-receiver"* Problem Frame is a problem pattern which reflects the need of distributing information between different users or applications by introducing an intermediate forwarder-receiver component. This component handles the communication. A new domain **bluetooth** could be identified by a further analysis of the chat application domain. New facts about the application domain can be collected in the domain knowledge for the chat software.

`F2:` The chat application shall run on mobile devices.

`F3:` Mobile devices communicate via a bluetooth connection.

This new domain knowledge indicates that collecting facts and assumptions about the application domain is an important aspect. If the software development goal (system mission and requirements) would be to develop a chat application for a desktop PC with an Internet connection, the new forwarder-receiver domain would be a W-LAN connection or the usage of TCP/IP, etc. In this case a client-server architecture would possibly fit better to the given properties of the application domain. In our application example we want that a chat application for mobile devices shall be developed so that a bluetooth domain will be responsible to administrate the forwarder-receiver tasks.

The *"forwarder-receiver"* Problem Frame in Fig. 10 describes the following problem situation: Based on an *operator* (**user**) command (`E3`: sendTextmessage) and the actual state of the *machine* (`chat application`) indicated by `Y1` phenomena controlled by *workpiece models* (here: content of actual phrased **text-message**) the machine requests the *forwarder-receiver* domain (**bluetooth**) to call `C2` operations: sendMsgText-services corresponding to the `E3` command. In its role of a forwarder the domain forwarder/receiver is a kind of required interface which initialize operation calls. In its role of a receiver the domain forwarder/receiver is a kind of provided interface which implements the behaviour of the called operations. If a forwarder/receiver gets a message (a service of the receiver is used by a forwarder) it forwards this `C4` command (receiveMsgText) to the machine which processes the corresponding `E1` operations (registerTextmessage).

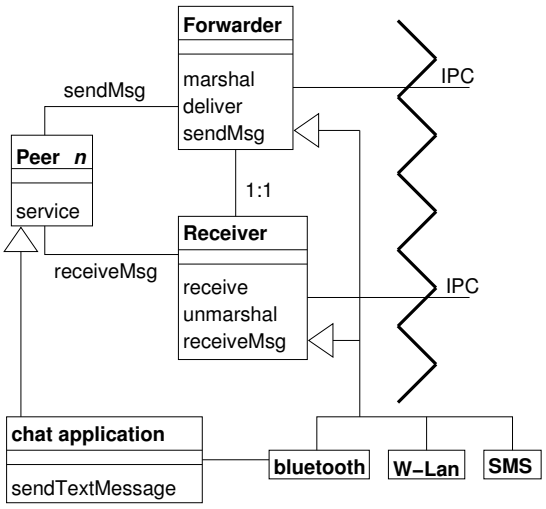


Figure 11: UML class diagram for the Design Pattern *"forwarder-receiver"* assigned with content of the PF in Fig. 10

Thus we achieve a peer-to-peer chat application in which the machine can be client and server depending on the corresponding service which is processed. Again there is no need to change the original design pattern *"forwarder-receiver"* of Buschmann et al. [5]. The content of the Problem Frame

in Fig. 10 can be combined with the design pattern in Fig. 11 by a simple inheritance relation. After the text-message is send to all the other users of the chat the Problem Frame in Fig. 5 is responsible to display the new, incoming text-message by an updated course of chat on the private user displays.

Like the Problem Frames of the analysis phase the two instantiated solution patterns of software design namely the architectural style *"model-view-controller"* in Fig. 9 and the design pattern *"forwarder-receiver"* in Fig. 11 are independent (sub)solutions to the given problem situation. These concurrent solution representations finally have to be merged in order to describe a design that represents a near code level description of the software which is to realise. To compose the overall design out of the parallel (sub)solutions we use UML sequence diagrams.

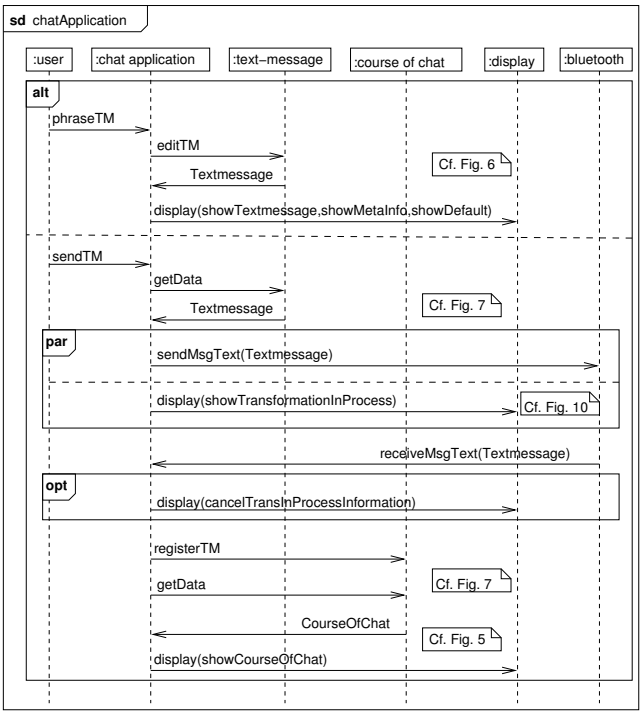


Figure 12: UML sequence diagram to specify the chat application software

The UML sequence diagram in Fig. 12 is derived with the help of the Problem Frames in Fig. 5, Fig. 6, Fig. 7, and Fig. 10. Domains of the Problem Frames are now objects of the sequence diagrams. The messages in Fig. 12 are taken from the UML class diagrams of Fig. 9 and Fig. 11. Remember they were derived out of the shared phenomena from the corresponding Problem Frames. The requirements `R1-R6` were used to define the time order of the messages in the sequence diagram. Now we have a specification of the chat application machine which can easily be used to develop the software.

5. CONCLUSIONS AND PERSPECTIVES

The integration of patterns from different computer science disciplines allows us to transform natural-language problem descriptions systematically into solution descriptions which are close to the software implementation level. For this we employ redundant or comparable pattern descriptions from HCI and SE to realize a pattern-based software development process. Problem Frames are used to represent a software development problem which is to be solved. Architectural styles and design patterns are used to reflect possible solutions of these software development problems. Based on an application example we show how Problem Frames as problem patterns can be transformed into architectural styles and design patterns as representatives for solution patterns to finally implement given software requirements.

Besides the systematic transformation of software artefacts of the analysis phase with software artefacts of the design phase we explicitly support refering non-functional software properties und improving the application of HCI design patterns within the overall software development process. Therefore we integrate the problem descriptions of HCI design pattern into Problem Frames which are use through requirements engineering. By this we achieve a possibility to describe software quality attributes like usability already and explicit in early software development phases. Nevertheless we remain in natural-language, HCI-oriented problem representations understandable by expert and laymen.Besides the core functionality of the software to be developed HCI-oriented Problem Frames refering software quality aspects like usability and can systematically transformed into architectural styles and design patterns. For the application example we demonstrated the qualitative improvement of the desired functionality.

A problem pattern as in Fig. 8 together with its corresponding solution pattern in Fig. 9 represent a pattern case which can be stored into a pattern case base and reused by CBR techniques to solve different problem situations. Future tool support for this transformation of problem into solution patterns is planed. CBR techniques could also make suggestions which of the possible solutions is the most appropriate one for a given application example and for that reason should be applied. Currently, we extend the chat application example and analyse the relevance of the domain knowledge for the choice of patterns and its role for the appraisal of software quality characteristics.

The applied problem decomposition technique using knowledge-based projection eases the later, futher development of the application example in an evolutionary style. New requirements are assigned to already existing subproblems or a new Problem Frame will represent the problem situation the new requirement is involved to. In contrast to use-cases Problem Frame explicitly show the translation of a natural-language requirement into interactions of the machine with the corresponding entities of the environment. A Problem Frame can be translated into objects and messages of a UML class or sequence diagram. Starting with Jackson diagrams like context diagram and Problem Frames in the analysis phase provides the distinction of natural-language problem descriptions to (semi)-formal, technical (for instance, UML) representations of a detailed solution to a given problem in the design phase. Whereas a deterministical transformation of a Problem Frame into a UML diagram is possible.

Furthermore we are interested in investigating how HCI-

oriented Problem Frames can be combined with GUI design elements (like buttons, check boxes, text field, etc.) to assist the design of user-friendly applications. Fig. 3 for instance could usefully implemented by a labeled text field widget with a default text shown on initialization time of the text field.Based on this finding we will try to transform the coarse-grained design description represented by architectural styles and the design patterns we have use to a more detailed design which could result into concrete design components like GUI widgets.

6. REFERENCES

[1] A. Aaamondt and E. Plaza. Cased-Based Resoning: Foundational Issues, Methodological Variations, and System Approaches. *AI Communications*, 7(1):31–59, 1994.

[2] C. Alexander, S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, and S. Angel. *A Pattern Language.* Oxford University Press, New York, USA, 1977.

[3] S. Bjork and J. Holopainen. *Patterns in Game Design.* Charles River Media, Hingham, USA, 2005.

[4] J. Borchers. *A Pattern Approach to Interaction Design.* John Wiley & Sons, Chichester, USA, 2001.

[5] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal. *Pattern-Oriented Software Architecture – A System of Patterns.* John Wiley & Sons, Chichester, USA, 1996.

[6] C. Choppy and M. Heisel. Une approache á base de patrons pour la spécification et le développement de systèmes d'information. *Approches Formelles dans l'Assistance au Développement de Logiciels - AFADL*, 2004.

[7] L. Chung, B. A. Nixon, E. Yu, and J. Mylopoulus. *Non-Functional Requirements in Software Engineering.* Kluwer Academic Publishers, Boston, USA, 2000.

[8] E. Folmer, M. van Welie, and J. Bosch. Bridging Patterns: An approach to bridge gaps between SE and HCI. *Information and Software Technology*, 48(2):69–98, 2006.

[9] M. Fowler. *Analysis Patterns: reusable object models.* Addison-Wesley, Boston, USA, 1996.

[10] E. Gamma, R. Helm, R. E. Johnson, and J. Vlissidis. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley Professional, Boston, USA, 1995.

[11] M. Jackson. *Problem Frames. Analyzing and structuring software development problems.* Addison-Wesley, Boston, USA, 2001.

[12] L. Rapanotti, J. G. Hall, M. Jackson, and B. Nuseibeh. Architecture-driven Problem Decomposition. *Proceedings of the 2004 International Conference on Requirements Engineering (RE04), Kyoto*, 2005.

[13] S. Robertson and J. Robertson. *Mastering the Requirements Process.* Addison-Wesley, Boston, USA, 1999.

[14] M. Shaw and S. Garlan. *Software Architecture. Perspectives on an Emerging Discipline.* Prentice Hall, Eaglewood Cliffs, New Jersey, USA, 1996.

[15] D. Sinnig, P. Forbrig, and A. Seffah. Patterns in model-based development. In H. Trætteberg, J. Molina, and N. J. Nunes, editors, *First International Workshop of MBUI. CEUR Workshop Proceedings*, 2004.

[16] W. F. Tichy. A catalogue of general purpose design patterns. In *Proceedings of Technology of Object-Oriented Languages and Systems (TOOLS23).* IEEE Computer Society, 1998.

[17] J. Tidwell. *Designing Interfaces.* O'Reilly Media, Sebastopol, USA, 2005.

[18] M. van Welie. Patterns in interaction design, 2003-2006. http://www.welie.com/ Online catalogue for interaction design patterns.